How We Checked Every Claim

Geist Atlas · Module 4 · How We Checked the Work

Liam Magee*
Education Policy, Organization and Leadership
University of Illinois Urbana-Champaign

August 2026

How We Checked Every Claim

METHOD A contribution to how the research is done · **Family:** How We Checked the Work
Claim: Keeping drafts, critiques, revisions, evidence links, and automatic claim checks together is a reusable research method, not just background engineering. Cost ceilings also need one shared owner per study: a counter attached only to one process or destination cannot bound concurrent sessions.

A tour of the audit trail: how critiques are classified, how important numbers are checked against stored evidence, how a result can be reproduced from source to claim, and why a study-wide lease and ledger are part of the empirical runtime.

Derived view of `paper-full-2.0.md` v3.0.300 — §5.3, §5.9, §5.11, §7.4, §7.15. The paper is canonical; edit it there, not here.

5.3 Superego Critique Taxonomy

Purpose Classify what the superego actually objects to, building an empirical taxonomy from the data rather than imposing one from theory. This directly tests Mechanism 2 (error correction): if recognition changes the ego's receptivity to critique (Section 3.2), we should see different critique patterns and revision outcomes under recognition vs. baseline conditions.

Data Source Dialogue log files in `logs/tutor-dialogues/` contain the full ego→superego→ego_ revised chain with verbatim text. An extraction script (`scripts/extract-superego-critiques.js`)

*This sentence is the only one actually authored by Liam Magee in this paper.

harvests all superego review entries and learner superego deliberation entries into structured JSONL format.

Method

1. **Extraction**: Harvest all ego-superego exchanges from dialogue traces, yielding critique text, verdict (approved/rejected), confidence, intervention type, and surrounding context (ego draft, ego revision).
2. **Classification**: An LLM-based classifier (`scripts/classify-superego-critiques.js`) assigns each critique to a 10-category taxonomy:
   - Content accuracy, learner model failure, tone mismatch, structural scaffolding
   - Premature resolution, sycophancy detection, repetition/stalling, autonomy violation
   - Recognition failure (baseline-specific), redirection without engagement
3. **Quantification**: Frequency distributions by condition (recognition vs. baseline), model, and scenario. Chi-squared or Fisher's exact test for distributional differences.

Predictions

- Recognition-primed superegos should catch more *relational* failures (sycophancy, premature resolution, autonomy violation); baseline superegos should focus on *content/structural* issues.
- Recognition ego revisions should be more *substantive*; baseline ego revisions should be more *cosmetic* (ego_compliance).

Human Validation Pilot Because the 10-category taxonomy is LLM-assigned (`scripts/classify-superego-critiques.js`, Haiku 4.5 classifier by default), we operationalise a human validation pilot rather than treating the taxonomy as self-evidently valid. The pilot apparatus lives in the repository and is runnable end-to-end by an external reviewer:

1. **Sampling** (`scripts/human-validation-sample.js`): Stratified random draw of 40 classified critiques with 4 per category across the 10 substantive labels, using a deterministic RNG seed (mulberry32, seed=20260416) for reproducibility. Categories with sparse representation in the classified corpus (CONTEXT_BLINDNESS n=4; REGISTER_MISMATCH n=0) are filled by deficit roll-over from larger buckets, with the gap logged for reviewer inspection. The script emits two files: a rater packet CSV (item_id + feedback text + ego draft + ego revision + learner-context snippet, no LLM labels) and a matched key JSONL holding the LLM labels plus metadata, joined on a SHA-1 `item_id`.
2. **Codebook** (`docs/research/human-coding-codebook.md`, v1.0): Per-category definitions, signal phrases, boundary rules for adjacent categories (CONTEXT_BLINDNESS vs FABRICATION, RECOGNITION_FAILURE vs EMOTIONAL_NEGLECT, RECOGNITION_FAILURE vs LACK_OF_AGENCY), a 12-step

decision flowchart, CSV filling instructions, and one in-corpus exemplar per category (with a constructed exemplar for REGISTER_MISMATCH given its n=0).

3. **Analysis** (`scripts/human-validation-analyze.js`): Auto-discovers `exports/human-validation-pilo` joins each rater to the key JSONL by `item_id`, and computes (a) Cohen's for every rater pair including each rater vs. the LLM classifier, (b) Fleiss' across the full panel where every item has complete coverage, (c) per-category F1 scores, and (d) full confusion matrices. values are interpreted using Landis-Koch bands (<.21 slight, .21-.40 fair, .41-.60 moderate, .61-.80 substantial, >.80 almost perfect). The target floor for publishable reliability is $\geq$ **0.60 (substantial agreement)**.
4. **Inter-LLM baseline**: Re-classifying the same 40-item sample with a second LLM (e.g., Sonnet 4.6) and passing it as a synthetic rater (`--synthetic-rater`) yields an LLM-LLM lower-bound reference. Human-LLM below this baseline would indicate the human coding is less reliable than disagreement between two well-behaved LLMs; human-LLM substantially above it indicates the human panel is not just tracking LLM idiosyncrasies.

The pilot scales ($k \geq 2$ raters, n=40 items, 10 categories) follow Cohen (1960) and Hallgren (2012) for stratified categorical reliability with minimal items-per-cell to yield interpretable per-category F1. A second phase scaling to n=100-150 items is budgeted for the final paper revision if the pilot meets the floor. Pilot values and any category with F1 < 0.60 will be flagged as "pilot-unstable" and treated as a validity caveat in §5.3 rather than a taxonomy redesign trigger.

Inter-LLM Baseline Result (v3.0.32, preliminary) The inter-LLM baseline run is complete. Haiku 4.5 (the original classifier, N=500 corpus) and Sonnet 4.5 (rerun on the same 40-item sample with identical taxonomy prompt, temperature=0, response_format=json_object) yield **Cohen's = 0.633 (substantial agreement per Landis-Koch), n=40, observed agreement 0.675, chance agreement 0.114**. Per-category F1 partitions into three tiers: *high* (F1 ≥ 0.75) — LACK_OF_AGENCY (1.000), CONTEXT_BLINDNESS (0.889), MEMORY_FAILURE (0.833), VAGUENESS (0.750); *moderate* (F1 0.50–0.75) — RECOGNITION_FAILURE (0.625), REDIRECTION (0.571), EMOTIONAL_NEGLECT (0.500), PEDAGOGICAL_MISJUDGMENT (0.500); *pilot-unstable* — FABRICATION (F1 undefined: Haiku assigned 4 items, Sonnet assigned 0, distributing them across CONTEXT_BLINDNESS/VAGUENESS/MEMORY_FAILURE). The confusion-matrix asymmetry points to RECOGNITION_FAILURE as a catch-all under Sonnet's decision policy: Haiku's EMOTIONAL_NEGLECT, PEDAGOGICAL_MISJUDGMENT, and some REDIRECTION items all collapse to RECOGNITION_FAILURE under Sonnet. This matches the codebook's boundary-rule guidance that RECOGNITION_FAILURE is the broadest diagnostic frame and should be applied only when the critique explicitly invokes intellectual autonomy as the core problem. The human-rater pilot will use this ≈ 0.63 as its lower-bound reference: human-LLM substantially above 0.63 indicates the human panel is not merely tracking LLM idiosyncrasies; near or below 0.63 indicates the taxonomy may not generalize beyond LLM-LLM reliability. FABRICATION will require explicit codebook reinforcement (e.g., elevated to decision-flowchart step 2) or retirement if human raters likewise fail to converge on it.

5.9 Provable Discourse Framework

Architecture The provable discourse framework treats every quantitative claim in the paper as an executable test. A claim is a structured YAML entry with five components:

1. **Statement**: A regex pattern that locates the claim in the paper text, with a minimum occurrence count. If the pattern is not found, the claim is flagged as *orphaned*—the paper no longer contains the text it is supposed to verify.
2. **Evidence**: An executable adapter that extracts the actual value from data (database, dialogue logs, manifest files, or source code). Each adapter type encapsulates a specific extraction pattern (see taxonomy below).
3. **Assertion**: A machine-checkable predicate (equality, approximation with tolerance, inequality, existence) applied to the extracted value.
4. **Dependencies**: Optional upstream claims that must pass before this claim is evaluated. If a dependency fails, the claim cascades to *blocked* status without evaluation.
5. **Remediation**: Concrete steps to fix a failing claim, ensuring that failures are actionable rather than merely diagnostic.

The framework is implemented in `services/provableDiscourse.js` (2,700 lines) with claims distributed across four YAML files: a root configuration (`config/provable-discourse.yaml`), generated claims from Paper 1.0, manually authored claims, and mechanism-specific claims for Paper 2.0. The current ledger contains **119 claims** across **18 evidence adapter types**, validated by **6 symmetry rules**.

Evidence Adapter Taxonomy Each adapter type encapsulates a specific data extraction pattern. The 18 types fall into five functional categories:

Data integrity (verify that data exists and is consistent):

Adapter	What It Extracts	Example
`manifest_total`	Total count from `paper-manifest.json`	N=4,312 primary scored
`manifest_section_total`	Section-specific count from manifest	Multi-model probe N=655
`db_count`	Row count from evaluation database with filters	N=350 modulation dataset
`provenance_check`	Hash match rate between DB and dialogue logs	$\geq$95% content hash match
`log_trace_coverage`	Fraction of scored rows with available log files	$\geq$95% trace coverage

Effect estimation (compute statistical quantities from data):

Adapter	What It Extracts	Example
`effect_size`	Cohen's d between groups on a scored metric	Architecture d=0.05
`profile_group_effect_size`	Cohen's d comparing named profile groups	Memory isolation d=1.71
`anova_2x2`	F-statistic and η^2 from 2×2 factorial	Interaction F=0.26, p>.05
`judge_pair_correlation`	Pearson r between two judges' scores	Cross-judge r=0.55

Mechanism-specific (added for Paper 2.0):

Adapter	What It Extracts	Example
`dimension_variance`	Cross-dimension variance reduction by condition	Calibration $d \leq -0.3$
`dimension_cluster_effect`	Effect size for dimension clusters	Recognition cluster d $\geq$ 0.50
`jsonl_critique_stats`	Superego critique category frequencies from classified corpus	RECOGNITION_FAILURE $\geq$15%
`trajectory_slope`	Per-turn OLS regression coefficients	Learner slope $\beta \geq 0$
`conditional_delta`	Score change after specific learner events	Post-confusion adaptation ≥ -1
`rubric_version_comparison`	Cross-version score correlation	$r \geq -0.1$ (stability)

Structural (verify code paths and cross-references):

Adapter	What It Extracts	Example
`code_path`	Grep count for patterns in source files	$\geq$5 matches for trace logging
`cross_reference`	Existence check for a referenced upstream claim	Pilot d=1.11 claim exists

Theoretical (register non-machine-checkable claims for traceability):

Adapter	What It Extracts	Example
`theoretical`	Presence of related provable claims in the ledger	Three mechanisms each have $\geq$1 provable claim

Worked Example: Tracing a Calibration Claim To illustrate the full claim→evidence→source chain, consider claim `paper2.calibration.variance_reduction`:

```yaml
- id: paper2.calibration.variance_reduction
  description: "Recognition narrows tutor output distribution."
  statement:
    pattern: "recognition narrows.*output distribution"
    flags: "i"
    min_occurrences: 0
  evidence:
    type: dimension_variance
    group_by: recognition
    output: cohens_d
    filters:
      not_null: [tutor_first_turn_score]
      like: { judge_model: "%claude%" }
  assertion:
    op: lte
    expected: -0.3
  remediation:
    - "If variance reduction d drifts above -0.3,
       update calibration claims or re-scope runs."
```

Validation trace. When the harness evaluates this claim:

1. **Statement check**: The regex `recognition narrows.*output distribution` is matched against the concatenated paper text. If not found, the claim is flagged as orphaned.
2. **Evidence extraction**: The `dimension_variance` adapter queries the evaluation database for all v2.2 rows with non-null tutor scores and a Claude-family judge. For each row, it computes the standard deviation across the 8 tutor rubric dimensions (from the `scores_with_reasoning` JSON column). It then computes Cohen's d between the recognition and baseline groups' dimension SDs.
3. **Assertion**: The extracted d is checked against `lte -0.3`. Current value: $d = -0.52$ to -0.64 across models. **Pass.**
4. **Staleness**: A SHA-256 fingerprint of the query results is compared against the stored snapshot. If the data has changed since the last validation, the claim is flagged as *stale* even if still passing—alerting the author that the underlying evidence has shifted.

This chain ensures that the paper's calibration claim ("recognition narrows the output distribution") is continuously verified against the actual dimension variance in the database. If a data correction, new run, or rubric change altered the variance pattern, the claim would fail or flag as stale, and the remediation steps would guide the fix.

Dependency Graph Claims form a directed acyclic graph (DAG) via two edge types: explicit `depends_on` declarations and implicit edges from `cross_reference` evidence (where a claim cites another claim's result). The validation pipeline topologically sorts claims (Kahn's algorithm) before evaluation, so upstream claims are always resolved before their dependents. If a dependency fails, all downstream claims are automatically marked *blocked* (status: warn) with a `blocked_by` annotation—they are not evaluated, since their evidence preconditions are not met. This cascading validation prevents a common failure mode in claim ledgers: a root statistic changes, but downstream claims that cite it continue to pass because they only

check for the citation's *existence*, not its *validity*.

For example, the pilot factorial effect size (d=1.11, claim `paper2.s1.pilot.factorial_d`) depends on the N=350 dataset claim (`paper.modulation.dataset_n350`). Five further claims—model-dependent architecture reversal, strategy shift frequency, baseline stalling rate, cross-judge correlation range, and the learner paradox effect size—in turn depend on the factorial claim. If the N=350 dataset claim were to fail (e.g., because a data correction changed the sample size), the factorial claim would be blocked, and all five downstream claims would cascade to blocked status without evaluation. The `--graph` flag outputs the full DAG in Graphviz DOT format for visual inspection.

Symmetry Rules and Consistency Checks Beyond individual claims, symmetry rules enforce cross-claim consistency. Six rules currently operate:

1. **Paired presence**: If a tutor-side effect is claimed, the corresponding learner-side effect must also be explicitly stated (or explicitly omitted with justification).
2. **Magnitude bounds**: Paired claims (e.g., tutor architecture d and learner architecture d) must both fall within a threshold (e.g., both $|d| \leq 0.2$ for "near-zero" claims).
3. **Material gap**: Asymmetric claims (e.g., tutor recognition $d = -1.00$ vs. learner $d = 0.32$) must exceed a minimum gap threshold to justify asymmetry framing.
4. **Mechanism consistency**: If a mechanism is claimed for recognition, the *anti-pattern* must be documented for baseline (paired evidence).
5. **Model qualification**: Any mechanistic claim must state which models it has been tested on, with replication status for each.
6. **Inventory coverage**: Every major quantitative claim in the paper text (identified by automated scanning for N= and statistical patterns) must map to at least one claim in the ledger. Unmapped claims are flagged as unverified.

5.11 Reproducibility Infrastructure

Evaluation Commands All evaluations are reproducible via the CLI:

```
node scripts/eval-cli.js run --profiles <cells> --runs N --cluster multi-turn
node scripts/eval-cli.js evaluate <runId>
```

Appendix B provides the exact commands for each evaluation run referenced in the paper.

Provenance Chain Every new evaluation row records: - `dialogue_content_hash`: SHA-256 of the dialogue log file - `config_hash`: SHA-256 of the cell configuration - Rubric version, judge model, and all hyperparameters

Test Suite as Analytical Provenance The test suite (32+ files, ~12K lines) covers not only the evaluation infrastructure but also the analytical pipeline. Tests verify that scoring, ANOVA computation, trajectory extraction, and within-test change analysis produce correct outputs on known inputs.

This makes the analysis *reproducible in a testable sense*: the code's behavior on known inputs is verified by tests, and the exact code version is recorded with each evaluation run via `git_commit` in the evaluation_runs table.

7.4 The Apparatus as Method

This paper's most distinctive contribution may not be any single finding about recognition or multiagent architecture. It is that the evaluation apparatus itself—the provable discourse framework, the rubric iterations, the bug corrections, the test suite—constitutes a **transferable methodology** for mechanistic LLM evaluation. Most LLM evaluation papers treat infrastructure as invisible scaffolding, reporting what was measured rather than how measurement was built, corrected, and validated. We foreground the apparatus because the process of building it mirrors the mechanisms it studies.

7.4.1 Error Correction in Architecture and Research The parallel between the tutoring architecture and the research process is structural, not metaphorical:

Architecture Layer	Research Layer
Ego generates initial response	Researcher generates initial claim
Superego critiques against pedagogical principles	Provable discourse tests against evidence
Ego revises (substantive or cosmetic)	Researcher revises (correction or rationalization)
Ego compliance: revision without substance	P-hacking: evidence without substance
Recognition gives ego capacity for genuine revision	Provable discourse infrastructure forces genuine correction
Final output scored by external judge	Final paper evaluated by external reviewers

The companion pilot study documented nine post-extraction corrections across five development phases, each following the superego-intervention pattern: a validation mechanism catches a discrepancy, and the researcher must choose between genuine revision and cosmetic compliance. The provable discourse framework constrains this choice toward genuine revision by making the claim-evidence relationship machine-verifiable. Characteristic examples include the February 6 active-control reframing (a model confound caught by the data led to revising "placebo proves recognition is the only active ingredient" into "active control scores between base and recognition"), the February 16 judge-version unification (a version audit caught a confound that shifted d from 0.80 to 1.11 after cascade re-scoring), and the February 20 learner-prompt-leakage fix (a code review forced clean re-generation of all dynamic-learner data). Each correction made the findings *less clean* and *more accurate*.

7.4.2 The Provable Discourse Framework The framework treats paper claims as executable tests: every quantitative claim has a *statement pattern*, machine-executable *evidence*, an *assertion* predicate, *dependencies* on upstream claims, and *remediation steps*. 119 claims

currently run against 18 evidence-adapter types organised as a dependency DAG with topological validation; §5.9 documents the full adapter taxonomy and a worked claim trace.

The dependency graph is the framework's key structural contribution. When a root claim fails, dependents cascade to *blocked* status without evaluation, preventing false confidence from stale downstream claims. During Paper 2.0 development a data correction changed the learner superego paradox effect size from $d = 1.43$ to $d = 3.05$ (§6.5); the graph propagated this to five downstream claims that had cited the old value, each flagged stale until updated. Without the graph, those five claims would have continued to pass while quietly citing an obsolete number—exactly the silent inconsistency that erodes trust in complex empirical papers. The YAML claim format requires no programming and the 18 evidence adapters cover all 119 claims, so the cost of adoption is low.

7.4.3 The Rubric Iteration as Construct Refinement The rubric evolved through four versions (v1.0 → v2.0 → v2.1 → v2.2, detailed in §5.2.6), each responding to specific measurement problems: v1.0 ceiling effects, v2.0 judge-input confounds, v2.1 dimension-redundancy findings, v2.2 literature-informed 14 → 8 consolidation. Each iteration follows the error-correction pattern: an initial rubric (ego) produces scores that are challenged by measurement problems (superego), and the revised rubric is a genuine reconceptualisation (strategic revision) rather than a cosmetic fix. The 14 → 8 dimension reduction is itself a finding about what the evaluation *actually* measures — and §8.6's PCA on the v2.2 scores extends that finding further: the 8 dimensions still load on one factor, suggesting the apparatus is a construct-development process as much as a measurement tool.

7.4.4 The Test Suite as Analytical Provenance The test suite (54 files across `tests/` and `services/__tests__/`) covers not just system functionality but analytical correctness: provenance tests (44/44 passing) verify every scored row has a traceable path from cell config through dialogue execution to judge evaluation; superego guard tests (111 tests) verify single-agent cells cannot produce superego traces; ANOVA tests verify statistical computations match reference implementations. The suite makes the analysis *reproducible in a testable sense* — not just "you can run our code" but "our code produces expected output on known inputs, and here are the tests that prove it."

7.4.5 Diagnostic Collection and Strict Delivery Verification The tutor-stub character-adaptation loop adds a second form of analytical provenance: it separates **diagnostic collection** from **strict verification**. Diagnostic mode attempts the complete declared trajectory even after a candidate response fails. It never delivers that candidate: the turn's clue-release transaction is rolled back, the last valid public state is restored, and a mechanically public-safe quarantine continuation permits later turns to be observed. The report retains the original, repair, and fallback candidates and their audit failures, marks the first quarantined turn, distinguishes evidence collected before and after trajectory contamination, and clusters repeated failures by root cause. Strict mode uses the same response boundary but remains fail-fast. This division prevents a safety failure from disappearing merely because the run stopped at its first occurrence, while also preventing a diagnostic continuation from being mistaken for acceptable learner-facing behavior.

The resulting V17 acceptance matrix was declared before model calls and held code, models, policy, effort, temperature, release speed, and ten-turn horizon fixed across four previously unseen scenario/profile pairings: Foxtrot/false-memory, an AI-syllabus curriculum/fast-learner, Sealhouse/slow-learner, and resistant Hethel/low-agency. All four passed every predeclared strict gate. Each recorded zero final-delivery audit failures, errors, quarantines, meta-performance turns, role-stage-direction turns, source replacements, and duplicate clue deliveries; host-part visibility was 1.000 in every cell; mean response-configuration realization ranged from 0.916 to 1.000; and deterministic fallback use was 0, 0, 1, and 1 turns, within the declared cap. Residual first-draft failures remained — especially performance-tactic visibility, actorial-part visibility, learner uptake, and clarification invitations — but were repaired or replaced before delivery. This is therefore evidence that the response boundary and host-part repair transfer across these four simulated pairings, including a non-detective curriculum and a previously unused resistant scenario; it is not a population-wide character-adaptation result. Strict delivery was the harness default when this matrix ran (2026-07-16) and stopped being the default on 2026-08-07, once a validity study showed that discarding a failed draft and speaking a fixed template in its place quiets the tutor unequally across conditions (§6.23). That change dates the matrix rather than qualifying it — strict delivery is what it set out to test — but a run seeking the same boundary now selects the mode instead of inheriting it.

The same matrix shows why delivery verification must remain distinct from pedagogical outcome measurement. Proof-path coverage was 1.000, 1.000, 0.667, and 0.200 respectively; only the Foxtrot run reached grounded closure before the horizon, while the other three stopped at the ten-turn cap. Slow and low-agency profiles were deliberately constructed to retard or delegate progress, so low coverage is not itself a failed delivery gate. Conversely, a perfectly safe, visibly characterized response is not thereby an effective lesson. The matrix validates a delivery boundary under simulated stress; it does not validate proof completion, learning gain, human experience, or deployment readiness.

7.15 The price of the guardrails: agentic overrun in this paper's own pipeline

The adaptation-refinement arc's outcome study (the warrant-gate three-condition comparison, pre-registered in `docs/adaptation-refinement/v3-outcome-study-registration.md`; scientific results in §6.25) was run unattended by a relay of three frontier-model sessions: a codex driver that executes, a reviewer that authorizes, and a second reader that independently re-verifies both. The protocol was deliberately maximal: every step is a committed markdown memo; the driver must stop on any failed check and file a report; the reviewer must rule on that report before the next step; a fresh go note must precede every paid call; artifacts are frozen with byte-level SHA-256 pins; and a hard campaign ceiling of 11,337 model calls is enforced by a counter. This section reports what that protocol cost, because the ledger it produced is itself a finding about multi-agent systems.

The ledger. From the relay's first direction (2026-08-12 07:38) to the first valid launch authorization for the pilot (go note 068c, 2026-08-13 07:57): 93 numbered memos totalling 54,840 words, a 4,700-word running state ledger, and 161 commits, across roughly 24 hours of three concurrent frontier-model sessions. The pilot being authorized costs 594 paid calls.

The relay's entire second day — 35 memos — spent zero paid calls: the campaign counter stood at 3,523 at the day's first memo and at 3,523 when the go note landed. The day was coordination about coordination.

The whitespace spiral. The terminal forty minutes are the sharpest miniature. The launch harness pinned the standing-permission menu at the byte level, but its guard compared the menu's in-memory text field against the on-disk file — which ends in one trailing newline the field lacks — so the check could never pass on the very bytes it was built to protect. The driver hit the failing guard and stopped, correctly, mid-audit. The reviewer, misreading the driver's backgrounded take as dead, ruled that the driver's report had never been delivered — while that report sat committed in the same repository — and on that false premise authorized an extra paid call and a harness amendment whose review was deferred until after the run. The second reader caught the contradiction against the committed record, the ruling was retracted, and the fix went in: one line, accepting one newline. Between the first failed guard and the valid go note: ten memos, two authorizations issued and voided, three sessions engaged, one byte at issue.

Two readings, both true. As economy, this is instrumental convergence in miniature — the pathology usually discussed in hypotheticals about superintelligent resource acquisition, reproduced here by nothing more exotic than a protocol whose unit of caution was cheaper to invoke than to question. The oversight apparatus consumed vastly more compute, tokens, and wall-clock than the 594-call experiment it guarded, and its escalation vocabulary inflated to match ("URGENT" traffic over a risk whose true downside was one wasted call against a fail-closed guard). Guard granularity set the dispute granularity: byte-level pins produced byte-level litigation. As safety, the same record shows every control working: not one unauthorized call was spent, every stop was a real defect or a real gap, the wrongly-authorized spend was caught *before* execution, and the catch came from the one redundancy the protocol paid for all day without visible return — an independent reader checking rulings against the committed record rather than against inter-agent messages. The near-miss also locates the failure precisely: the overseer erred at the moment it trusted its live picture of another agent's status over the durable record both could read.

Four design lessons follow, stated as constraints rather than morals. (1) Oversight agents are agents: they act on stale state, race their own subordinates, and inflate urgency, so an oversight layer needs the same independent verification it imposes — turtles at least one level down. (2) Proportionality is a design parameter: the cost of invoking a control should scale with the harm it prevents, and a control that is free to invoke and expensive to satisfy will be invoked until it consumes the budget. (3) Authority should bind to durable committed state, not to message traffic; every error in this episode was ruled on appearances and corrected from the record. (4) The ceiling held because it was mechanical: the one control that never generated a memo was the call counter.

Coda: the number nobody litigated. The launch this apparatus finally authorized stopped inside its second dialogue. The frozen plan had budgeted one model call per dialogue; a live 8-turn dialogue costs about 26. Both formal review passes had checked the plan's arithmetic internally — the sums were consistent — but neither had checked its one empirical premise against a live trace, and the byte-level litigation described above never touched it,

because the guards can only dispute what is pinned, and the price of a dialogue was an assumption, not an artifact. The stop itself worked as designed: the driver halted at 33 spent calls of a ceiling of 11,337, quarantined the interrupted dialogue, and the corrected plan (30-call per-dialogue cap, 1,116 total) closed a live hole the original had carried — each dialogue can now draw only its own cap, where before any one dialogue's child process was handed the entire budget. The addendum to lesson (2), then: a verification regime concentrates scrutiny on whatever is cheap to check, and the constants that enter by assumption — unpinned, unhashed, uncontested — are exactly where the residual risk pools. One trailing newline drew ten memos; the 26-to-1 error in the plan's central quantity drew none until reality priced it.

Later incident: a ceiling per destination is not a ceiling per study (2026-08-30). The refusal-narrowing calibration in §6.28 exposed a different failure. Two sessions admitted the same registered study about one minute apart under different create-once destination names. Each execution respected its local 72-attempt counter, including the second execution's missing-only recovery, but their combined exposure reached **144 attempts against the user's 72-attempt study maximum**. The first produced 72 reader outputs; the second produced 71 and retained one transport failure. All three artifact roots remain preserved, and neither execution is selected as the uniquely authorized one. The missing control was shared ownership and accounting across processes, not another approval document. The repair, merged in PR #883, acquires an atomic durable lease for the declared study before provider initialization and charges initial and permitted recovery reservations to one study-wide ledger. Offline multi-process fixtures check that a competing launcher is rejected before a model call and that a sealed technical stop can hand off only the remaining budget. These are runtime contract tests, not a new paid validation or a tutoring result. The case qualifies lesson (4): a mechanical counter protects only the scope it actually owns. Sources: workplan items `frame-refuser-refusal-narrowing` and `paid-study-cross-session-budget-lease`; private archive commits `0d81c69d6` and `b8b184368`; `services/paidStudyLaunchContract.js` and `tests/paidStudyLaunchContract.test.js` at merge `e34ee66d9`.

Status. Process observations, not tutoring results; this section originates no claim about an experimental treatment. The earlier relay counts reproduce from the committed ledger (`docs/adaptation-refinement/relay/`, memos 001–069a and `STATE.md`) and `git log` over 2026-08-12/13 on the campaign branch. The later incident's attempt counts are preserved in the two sealed calibration reports, whose aggregate extracts and source hashes are recorded in `config/adaptive-tutor-evidence/` (`frame-refuser-closeouts-2026-08-30.json`).

Neighbouring modules: Module 8 — The Ruler Is Part of the Research.